

Breaking the KAZ Suite: Practical Key Recovery Attacks on MySEAL 2.0’s Post-Quantum Candidates

Zhuo Huang¹, Chongrong Li¹, Yu Yu¹, and Xiaogang Zhou²

¹ Shanghai Jiao Tong University

² China Telecom Quantum Information Technology Group Co., Ltd

Abstract. We present practical attacks that completely break all four cryptographic schemes submitted to Malaysia’s MySEAL 2.0 standardization initiative: the KAZ-KA key agreement scheme, the KAZ-KEM key encapsulation mechanism, the KAZ-SIGN v1.6.4, and KAZ-SIGN v2.0 digital signature schemes. KAZ-KA, KAZ-KEM, and KAZ-SIGN v2.0 operate over $\mathbb{Z}_N$ where N is a primorial, the product of consecutive small primes. This design choice makes the group order $\varphi(N)$ extremely smooth, enabling efficient attacks. For KAZ-KA and KAZ-KEM, we recover the private key by enumerating candidates modulo each small prime factor and solving discrete logarithms in small groups. For KAZ-SIGN v2.0, we exploit the linear structure of signatures to formulate a hidden number problem instance, which we solve using lattice reduction with only two signatures. For KAZ-SIGN v1.6.4, we demonstrate universal signature forgery attacks using only the public key by exploiting its verification algorithm, without requiring the private key. All attacks are implemented and executed in under one second on a standard consumer laptop (a MacBook) across all suggested security levels (128, 192, and 256 bits). These results conclusively prove that the analyzed schemes are fundamentally insecure and unsuitable for any deployment or migration.

1 Introduction

The National Trusted Cryptographic Algorithm List (MySEAL) is an initiative by CyberSecurity Malaysia designed to establish a sovereign standard for cryptographic modules within the nation [12,13]. It serves to ensure that the cryptographic tools used for national security are robust and locally validated [12].

The initiative is divided into two primary categories:

1. AKSA (Algoritma Kriptografi Sedia Ada): Focuses on “Existing Cryptographic Algorithms” that are internationally recognized and already in use [12,13], such as AES, Camellia, PRESENT, Skinny, ChaCha20, SHA2/SHA3, PHOTON, BLS, ECDSA, HMAC, ASCON-AEAD.
2. AKBA (Anugerah Kripto Bestari Award): Targets “New Cryptographic Algorithms”, encouraging the development of indigenous or novel candidates to be vetted for future standardization [12,13].

The recent MySEAL 2.0 initiative was announced in March 2024, with a submission deadline set for March 7, 2025. Following this deadline, four candidate algorithms were submitted and have now entered the evaluation phase, commencing with the AKBA MySEAL 2.1 cycle in Q3 2025. These candidates are focused on asymmetric and post-quantum primitives [11] (with a demo provided at [26]):

1. KAZ-KA v1.0 [6]: A key agreement scheme that enables two parties to establish a shared secret over an insecure channel. The scheme operates over $\mathbb{Z}_{N_j}$ where N_j is the product of j consecutive odd primes $3, 5, 7, \dots, p_{j+1}$. The public key consists of two elements $A_1, A_2 \in \mathbb{Z}_{N_j}^*$ computed as $A_1 \equiv g_1^{a_1} \cdot g_2^{2a_2} \pmod{N_j}$ and $A_2 \equiv g_1^{a_2} \cdot g_2^{a_1} \pmod{N_j}$, where a_1, a_2 are private keys. The designers claim security based on the Double Discrete Logarithm Problem (DDLDP) and the Hidden Number Problem (HNP).
2. KAZ-KEM v1.0 [7]: A key encapsulation mechanism sharing the same mathematical structure as KAZ-KA. It uses the primorial modulus $N_j = \prod_{i=2}^{j+1} p_i$ and relies on the same underlying hardness assumptions. The encapsulation produces a ciphertext from which the shared key can be derived using the recipient's private key.
3. KAZ-SIGN v2.0 [5]: A redesigned digital signature scheme that returns to the primorial-based structure. Like KAZ-KA and KAZ-KEM, it operates over $\mathbb{Z}_{N_j}$ with N_j being the product of consecutive primes. However, unlike KAZ-KA and KAZ-KEM where the public key provides two equations in two unknown secrets, the public key of KAZ-SIGN v2.0 yields only a single equation. The scheme claims security from the DDLDP and the HNP.
4. KAZ-SIGN v1.6.4 [3]: A digital signature scheme based on an entirely different mathematical foundation. Unlike the other three schemes, KAZ-SIGN v1.6.4 relies on the Modular Reduction Problem rather than the discrete logarithm structure. The signature verification involves checking a congruence relation $V_1 \equiv \alpha \pmod{G_1 \rho}$ where G_1 is a large prime and ρ is derived from the message hash.

These algorithms are to be deployed in critical areas, including database encryption, next-generation firewalls/VPNs, and mobile radio networks, as companies actively seek broader domestic and international ecosystem partnerships [21, 19].

All four schemes are candidates in this cycle. KAZ-KA v1.0, KAZ-KEM v1.0, and KAZ-SIGN v2.0 share a common mathematical foundation: they operate over $\mathbb{Z}_{N_j}$ where $N_j = \prod_{i=2}^{j+1} p_i = 3 \cdot 5 \cdot 7 \cdots p_{j+1}$ is the product of j consecutive odd primes. The recommended parameters use $j = 65, 96, 122$ for 128-, 192-, and 256-bit security levels respectively. KAZ-SIGN v1.6.4 is fundamentally different, with details deferred to Section 5.

We also note that KAZ-SIGN v2.0 is distinct in nature from the NIST PQC Standardization Round 1 candidate KAZ-SIGN [4], which was broken by Bernstein via signature forgery attacks [8]. The NIST submission KAZ-SIGN used a “Second Order DLP” structure of the form $g^{R^\alpha} \equiv V \pmod{N}$, whereas KAZ-SIGN v2.0 employs the dual-generator structure $V = g_1^s \cdot g_2^t \pmod{N_j}$. Our work

focuses on key recovery for three schemes and public-key forgery for KAZ-SIGN v1.6.4.

In this paper, we show that all four schemes are insecure. For KAZ-KA, KAZ-KEM, and KAZ-SIGN v2.0, the root cause lies in the choice of the modulus N_j . Since N_j is a primorial, the group order $\varphi(N_j) = \prod_{i=2}^{j+1} (p_i - 1)$ factors completely into small primes. This extreme smoothness enables efficient application of the Pohlig–Hellman algorithm to solve discrete logarithms in the multiplicative group $\mathbb{Z}_{N_j}^*$.

For KAZ-KA and KAZ-KEM, we present direct key recovery attacks. Given only the public key, we can recover the private key by solving discrete logarithms modulo each prime factor of N_j and combining the results via the Chinese Remainder Theorem (CRT). These attacks complete within seconds on a standard laptop.

For KAZ-SIGN v2.0, a direct application of Pohlig–Hellman to the public key faces a practical obstacle: the algorithm requires storing discrete logarithm tables for each prime factor, which leads to prohibitive memory consumption at the recommended security levels. Instead, we develop a more practical lattice-based attack that exploits the HNP structure inherent in the signature scheme. By collecting only two valid signatures, we construct a lattice whose shortest vector reveals the signing key. This attack bypasses the memory bottleneck while maintaining computational efficiency. For KAZ-SIGN v1.6.4, we do not recover the signing key, but we can forge signatures from the public key.

The remainder of this paper is organized as follows. Section 3 reviews the mathematical background including the discrete logarithm problem, the Pohlig–Hellman algorithm, and lattice-based techniques for the HNP. Section 4 describes the KAZ schemes and their parameters. Section 5 presents our attacks on KAZ-KA, KAZ-KEM, KAZ-SIGN v1.6.4, and KAZ-SIGN v2.0. Section 6 concludes the paper.

2 Related Work

2.1 DLP in practice

Discrete logarithm computations have repeatedly exhibited gaps between conjectured theoretical hardness and deployed parameters. For finite-field Diffie–Hellman (DH) key exchange and DSA, the most acute operational risk arises when many servers reuse the same small prime modulus. The Logjam line of work [1,27] showed that after a one-time number field sieve precomputation for a widely used 512-bit prime, individual TLS/DH handshakes using that modulus can be solved efficiently enough to enable practical man-in-the-middle attacks at scale against legacy deployments. Beyond actively exploited settings, the public record for prime-field DLP computations—most notably the 768-bit prime-field discrete logarithm computation reported by Kleinjung *et al.* [17]—provides a pragmatic benchmark for parameter sizes plausibly within reach of well-resourced adversaries.

By contrast, elliptic-curve discrete logarithms on standard (non-pairing-friendly) curves still have no known comparable subexponential attacks; real-world breaks more often trace to poor curve selection or parameterization, such as curves susceptible to MOV [20] or Frey–Rück reductions that embed the problem back into a finite-field DLP. Consequently, standardized cryptographic algorithms and libraries emphasize the use of carefully selected base curves and strict validation to avoid such structural pitfalls.

Our work differs from the Logjam-style NFS attacks on large primes as follows: the MySEAL candidates instantiate their groups over a primordial modulus whose totient factors into small primes. This extreme smoothness makes the classical Pohlig–Hellman strategy the natural baseline, turning what appears to be a large-modulus DLP into many small-subgroup instances that can be solved and recombined efficiently.

2.2 HNP in practice

HNP provides a complementary lens: it models settings where an attacker observes noisy, truncated, or otherwise approximate modular relations involving a fixed secret, and reduces key recovery to lattice problems such as approximate shortest- or closest-vector instances. Boneh and Venkatesan [9] formalized this leakage-to-lattice pipeline while studying the bit security of Diffie–Hellman secrets, showing that sufficiently many imperfect observations can enable recovery of the hidden value. In real-world DSA and ECDSA deployments, this perspective explains how biased, repeated, or partially leaked per-signature nonces can lead to full private-key compromise without any advance in generic DLP algorithms. Howgrave-Graham and Smart [14] demonstrated that if a short window of consecutive nonce bits leaks (e.g., via side channels or partial state disclosure), then collecting enough signatures yields hidden-number instances that can be solved via LLL-based lattice reduction to recover the long-term signing key. Nguyen and Shparlinski [22] subsequently proved polynomial-time key recovery under mild randomness assumptions once a threshold number of signatures is available, strengthening earlier heuristic attacks with provable guarantees.

These ideas have repeatedly materialized in practice. The Minerva work [15] identified commodity smart cards and software stacks whose ECDSA scalar multiplication leaked only the nonce bit-length; despite this weak signal, the leakage still fits the HNP template, and lattice decoding enables recovery of 256-bit curve keys given sufficiently many signatures. Breitner and Heninger [10] further demonstrated a high-impact pipeline by collecting biased or repeated ECDSA signatures from cryptocurrencies and Internet-wide scans, and then automatically recovering large numbers of production private keys via HNP-style lattice attacks. Accordingly, modern engineering guidance emphasizes hardened nonce generation: either ensure strong, bias-free randomness with robust side-channel protections, or adopt deterministic nonce derivation (e.g., RFC 6979 [24]) to reduce exposure when entropy sources or hardware isolation are uncertain.

The MySEAL candidates introduce a designer-defined DDLP over $\mathbb{Z}_{N_j}^*$ with a primordial modulus, yielding algebraic structure in a group of high smoothness.

While DDLP is syntactically distinct from classical DLP or standard HNP formulations, our attacks follow a related methodological theme: we exploit parameter structure to reduce the problem into small-modulus components and recombine partial information to recover secrets efficiently. In particular, for KAZ-SIGN v2.0 we show that the signing equations themselves induce an HNP-like lattice instance, enabling practical key recovery without assuming any external leakage model.

3 Preliminaries

3.1 Notation

We write $\mathbb{Z}$ for the ring of integers and $\mathbb{R}$ for the field of real numbers. For a positive integer N , let

$$\mathbb{Z}_N := \mathbb{Z}/N\mathbb{Z},$$

denote the residue class ring modulo N . An element $a \in \mathbb{Z}_N$ is a *unit* (i.e., invertible) if there exists $b \in \mathbb{Z}_N$ such that $ab \equiv 1 \pmod{N}$. The set of all units forms a multiplicative group

$$\mathbb{Z}_N^* := \{a \in \mathbb{Z}_N : \gcd(a, N) = 1\},$$

called the *group of units* of $\mathbb{Z}_N$. In particular, when p is prime, $\mathbb{Z}_p$ is a field (canonically identified with $\mathbb{F}_p$), and $\mathbb{Z}_p^* = \mathbb{F}_p^\times$ is the multiplicative group of the field. For an integer $x \geq 1$, we write $\text{len}(x) := \lfloor \log_2 x \rfloor + 1$ for its bit-length.

Euler's totient function. The Euler totient function $\varphi(N)$ denotes the number of units modulo N , i.e., $|\mathbb{Z}_N^*| = \varphi(N)$. In particular, if p is prime and $k \geq 1$, then the units modulo p^k are exactly those residues not divisible by p , so $\varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1)$. Combining these two properties yields the standard formula for computing $\varphi(N)$ from the prime factorization of N . Let $N = \prod_{i=1}^t p_i^{e_i}$ be the prime-power decomposition of N (distinct primes p_i and integers $e_i \geq 1$). Then

$$\varphi(N) = \prod_{i=1}^t \varphi(p_i^{e_i}) = \prod_{i=1}^t p_i^{e_i-1} (p_i - 1).$$

Cyclic Groups. A group G is *cyclic* if it is generated by a single element $g \in G$, i.e.,

$$G = \langle g \rangle := \{g^k : k \in \mathbb{Z}\}.$$

For finite G , the *order* of g , denoted $\text{ord}(g)$, is the least positive integer t such that $g^t = e$. If $G = \langle g \rangle$ and $|G| = n$, then $\text{ord}(g) = n$ and every element of G can be written uniquely as g^k with some $k \in \{0, 1, \dots, n-1\}$. The group $\mathbb{F}_p^\times$ is cyclic of order $p-1$; a generator is often called a *primitive element*.

Discrete logarithm. Let $G = \langle g \rangle$ be a finite cyclic group of order n . For $x \in G$, the *discrete logarithm* of x to base g is the integer

$$\log_g(x) \in \mathbb{Z}_n,$$

satisfying $g^{\log_g(x)} = x$. When the base g is clear from context, we may simply write $\log(x)$.

Chinese Remainder Theorem (CRT). Let $N_1, \dots, N_k$ be pairwise coprime positive integers and set $N = \prod_{i=1}^k N_i$. The Chinese Remainder Theorem states that the natural reduction map

$$\begin{aligned} \mathbb{Z}_N &\longrightarrow \mathbb{Z}_{N_1} \times \dots \times \mathbb{Z}_{N_k} \\ x \bmod N &\longmapsto (x \bmod N_1, \dots, x \bmod N_k), \end{aligned}$$

is a ring isomorphism. Restricting to units yields a group isomorphism

$$\mathbb{Z}_N^* \cong \mathbb{Z}_{N_1}^* \times \dots \times \mathbb{Z}_{N_k}^*.$$

Equivalently, for any residues $a_i \in \mathbb{Z}_{N_i}$ there exists a unique $x \in \mathbb{Z}_N$ such that $x \equiv a_i \pmod{N_i}$ for all i . Specifically, let $M_i := N/N_i$, and since $\gcd(M_i, N_i) = 1$, let $t_i := M_i^{-1} \bmod N_i$. Then the unique x can be computed efficiently as

$$x \equiv \sum_{i=1}^k a_i \cdot M_i \cdot t_i \pmod{N}.$$

We therefore use the notation $\text{CRT}([a_1, \dots, a_k], [N_1, \dots, N_k])$ to denote this unique solution modulo N produced by the above construction.

Vectors and norms. Bold letters such as $\mathbf{v} = (v_1, \dots, v_n)$ denote vectors. We write $\|\mathbf{v}\|$ for the Euclidean norm,

$$\|\mathbf{v}\| := \sqrt{\sum_{i=1}^n v_i^2}.$$

3.2 Double Discrete Logarithm Problem

As defined in the KAZ-KA/KAZ-KEM/KAZ-SIGN v2.0 specifications, let N be a composite modulus and let $g_1, g_2 \in \mathbb{Z}_N^*$ be chosen so that neither $\log_{g_1}(g_2)$ nor $\log_{g_2}(g_1)$ exists modulo N . Let $\text{ord}(g_i)$ denote the order of g_i in $\mathbb{Z}_N^*$. Given

$$A \equiv g_1^s g_2^t \pmod{N},$$

the *Double Discrete Logarithm Problem* is to recover the pair $(s, t) \in \mathbb{Z}_{\text{ord}(g_1)} \times \mathbb{Z}_{\text{ord}(g_2)}$ from (A, g_1, g_2, N) . This is the hardness assumption claimed for KAZ-KA, KAZ-KEM, and later KAZ-SIGN v2.0 in their specifications. [6,7,5] The

documents also mention HNP as an auxiliary security layer; we treat DDLP as the primary assumption and focus on how the primorial modulus and smooth group order (next subsection) enable the reductions exploited in Section 5. They further suggest that DDLP/HNP could be potential quantum-resistant hard problems supporting a post-quantum candidate. [6,7,5]

3.3 Modular Reduction Problem (MRP)

As defined in the KAZ-SIGN v1.6.4 specification, let N be a composite modulus, let n be the bit-length of N , and let p_k be a factor of N . Choose α with $2^{n-1} < \alpha < N$ and compute

$$A \equiv \alpha \pmod{p_k}.$$

The *Modular Reduction Problem* is, given (A, N, p_k) , to recover α . This is the stated hardness assumption for KAZ-SIGN v1.6.4. [3] The specification likewise mentions HNP as an auxiliary layer; we treat MRP as the primary assumption and analyze its concrete instantiation in Section 5. The authors describe MRP as not well explored and suggest that it may be a potential quantum-resistant hard problem underpinning a post-quantum signature candidate. [3]

3.4 Primorials and Smooth Numbers

A positive integer n is called B -smooth if all prime factors of n are at most B . The primorial $p_k\#$ is defined as the product of the first k primes. In the KAZ schemes, the modulus is defined as $N_j = \prod_{i=2}^{j+1} p_i = 3 \cdot 5 \cdot 7 \cdots p_{j+1}$, the product of j consecutive odd primes.

For such a primorial N_j , the Euler totient function is

$$\varphi(N_j) = \prod_{i=2}^{j+1} (p_i - 1).$$

Since each $p_i - 1$ is even and factors into small primes, $\varphi(N_j)$ is extremely smooth. This property is critical to our attacks.

3.5 Discrete Logarithm Problem

The Discrete Logarithm Problem (DLP) in a cyclic group $G = \langle g \rangle$ of order n is defined as follows: given g and $h \in G$, find x such that $g^x = h$. The difficulty of DLP depends heavily on the structure of n .

The Pohlig–Hellman algorithm [23] exploits the factorization of the group order to reduce DLP to smaller subproblems. Let $n = \prod_{i=1}^k q_i^{e_i}$ be the prime factorization of the group order. The algorithm proceeds as follows:

1. For each prime power $q_i^{e_i}$ dividing n , compute the discrete logarithm modulo $q_i^{e_i}$ by working in the subgroup of order $q_i^{e_i}$.
2. Combine the results using the CRT to obtain $x \bmod n$.

The complexity of the Pohlig–Hellman algorithm is $O\left(\sum_{i=1}^k e_i(\log n + \sqrt{q_i})\right)$. When n is B -smooth, all $q_i \leq B$, making the algorithm efficient.

3.6 Lattices

A lattice $\mathcal{L}$ is a discrete additive subgroup of $\mathbb{R}^n$. Given a basis $\mathbf{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$, the lattice is

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}.$$

The Shortest Vector Problem (SVP) asks to find a nonzero vector in $\mathcal{L}$ of minimal Euclidean norm. The LLL algorithm [18] finds an approximate shortest vector in polynomial time. For a lattice of dimension n , the LLL algorithm outputs a vector $\mathbf{v}$ satisfying $\|\mathbf{v}\| \leq 2^{(n-1)/2} \lambda_1(\mathcal{L})$, where $\lambda_1(\mathcal{L})$ denotes the length of the shortest nonzero vector within the lattice $\mathcal{L}$. For random lattices, it is standard to use the Gaussian heuristic to estimate the scale of $\lambda_1(\mathcal{L})$ via $\text{gh}(\mathcal{L}) \approx (\det(\mathcal{L}))^{1/n} \sqrt{n/(2\pi e)}$ [2]. Here $\det(\mathcal{L})$ denotes the lattice determinant and n its dimension.

3.7 Hidden Number Problem

The Hidden Number Problem (HNP), introduced by Boneh and Venkatesan [9], asks to recover a hidden number α given partial information about $\alpha \cdot t_i \bmod p$ for known random values t_i . Formally, given d samples (t_i, u_i) where u_i approximates $\alpha \cdot t_i \bmod p$ with $|\alpha \cdot t_i - u_i \bmod p| < B$ for some bound B , the goal is to recover α .

HNP can be formulated as a Closest Vector Problem (CVP). Consider the lattice $\mathcal{L}$ spanned by the rows of

$$\mathbf{B} = \begin{pmatrix} p & 0 & \cdots & 0 & 0 \\ 0 & p & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & p & 0 \\ t_1 & t_2 & \cdots & t_d & 1 \end{pmatrix}.$$

The target vector is $\mathbf{t} = (u_1, u_2, \dots, u_d, 0)$. The lattice vector closest to $\mathbf{t}$ reveals α since $(\alpha t_1 \bmod p, \dots, \alpha t_d \bmod p, \alpha)$ lies in $\mathcal{L}$.

In practice, we reduce CVP to SVP using Kannan's embedding [16]. We construct the extended lattice $\mathcal{L}'$ spanned by the rows of

$$\mathbf{B}' = \begin{pmatrix} p & 0 & \cdots & 0 & 0 & 0 \\ 0 & p & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & p & 0 & 0 \\ t_1 & t_2 & \cdots & t_d & 1 & 0 \\ u_1 & u_2 & \cdots & u_d & 0 & K \end{pmatrix},$$

where K is a scaling factor. The target short vector is $(e_1, e_2, \dots, e_d, -\alpha, K)$, where $e_i = u_i - \alpha t_i \bmod p$ are small. Applying LLL to $\mathbf{B}'$ recovers this short vector, from which α can be extracted.

4 KAZ Schemes

We now describe the four KAZ schemes referenced in this paper. The first three schemes share a common structure: they operate over $\mathbb{Z}_{N_j}$ where $N_j = \prod_{i=2}^{j+1} p_i$ is the product of j consecutive odd primes, and use two selected primes $g_1, g_2 \in \mathbb{Z}_{N_j}^*$. In practice, g_1 and g_2 are chosen as odd primes and checked to satisfy $\gcd(g_i, N_j) = 1$. We denote by $o_1 = \text{ord}_{N_j}(g_1)$ and $o_2 = \text{ord}_{N_j}(g_2)$ the multiplicative orders of g_1 and g_2 in $\mathbb{Z}_{N_j}^*$. The remaining scheme, KAZ-SIGN v1.6.4, uses a different structure that we summarize separately below.

KAZ-KA v1.0. The protocol consists of three procedures. Algorithm 1 generates keys by sampling $a_1 \in \mathbb{Z}_{o_1}$ and $a_2 \in \mathbb{Z}_{o_2}$, then publishing A_1 and A_2 . Algorithm 2 uses fresh b_1 and b_2 with a slightly different structure to form the partial key B_1 and B_2 for exchange. Algorithm 3 combines the peer's partial key with the local secrets to compute the shared key. Correctness follows because both parties compute the same shared key: $B_1^{a_1} B_2^{a_2} \equiv A_1^{b_1} A_2^{b_2} \equiv g_1^{a_1 b_1 + a_2 b_2} g_2^{a_1 b_2 + a_2 b_1} \pmod{N_j}$.

Algorithm 1 KAZ-KA Key Generation Algorithm (Party A)

Require: $(g_1, g_2, o_1, o_2, N_j)$
Ensure: Public key (A_1, A_2) , private key (a_1, a_2)
1: Choose random $a_1 \in \mathbb{Z}_{o_1}$, $a_2 \in \mathbb{Z}_{o_2}$
2: $A_1 \leftarrow g_1^{a_1} g_2^{2a_2} \pmod{N_j}$
3: $A_2 \leftarrow g_1^{a_2} g_2^{a_1} \pmod{N_j}$
4: **return** $(A_1, A_2), (a_1, a_2)$

Algorithm 2 KAZ-KA Key Generation Algorithm (Party B)

Require: $(g_1, g_2, o_1, o_2, N_j)$, private key (a_1, a_2)
Ensure: Partial key (B_1, B_2)
1: Choose random $b_1 \in \mathbb{Z}_{o_1}$, $b_2 \in \mathbb{Z}_{o_2}$
2: $B_1 \leftarrow g_1^{b_1} g_2^{b_2} \pmod{N_j}$
3: $B_2 \leftarrow g_1^{b_2} g_2^{2b_1} \pmod{N_j}$
4: **return** (B_1, B_2)

Algorithm 3 KAZ-KA Full Key Agreement Algorithm by Parties A and B

Require: Partial key (A_1, A_2) and (B_1, B_2)
Ensure: Shared key K_A and K_B
1: Party A computes $K_A \leftarrow B_1^{a_1} B_2^{a_2} \pmod{N_j}$
2: Party B computes $K_B \leftarrow A_1^{b_1} A_2^{b_2} \pmod{N_j}$
3: **return** K_A and K_B , where $K_A = K_B$

KAZ-KEM v1.0. The key encapsulation mechanism uses the same structure as KAZ-KA for key generation. Algorithm 4 generates a public encapsulation key (e_1, e_2) from secrets $a_1 \in \mathbb{Z}_{o_1}$ and $a_2 \in \mathbb{Z}_{o_2}$. Algorithm 5 encapsulates a message by sampling fresh (b_1, b_2) , forming (B_1, B_2) , and masking m with $e_1^{b_1} e_2^{b_2}$. Algorithm 6 removes the mask with the decapsulation key by computing $m = c - B_1^{a_1} B_2^{a_2} \bmod N_j$.

Algorithm 4 KAZ-KEM Key Generation Algorithm

Require: $(g_1, g_2, o_1, o_2, N_j)$

Ensure: Public encapsulation key (e_1, e_2) , private decapsulation key (a_1, a_2)

- 1: Choose random $a_1 \in \mathbb{Z}_{o_1}$ and $a_2 \in \mathbb{Z}_{o_2}$
 - 2: $e_1 \leftarrow g_1^{a_1} g_2^{2a_2} \bmod N_j$
 - 3: $e_2 \leftarrow g_1^{a_2} g_2^{a_1} \bmod N_j$
 - 4: **return** $(e_1, e_2), (a_1, a_2)$
-

Algorithm 5 KAZ-KEM Key Encapsulation

Require: $(g_1, g_2, o_1, o_2, N_j)$, public key (e_1, e_2) , plaintext m

Ensure: Ciphertext $C = (B_1, B_2, c)$

- 1: Choose random $b_1 \in \mathbb{Z}_{o_1}$ and $b_2 \in \mathbb{Z}_{o_2}$
 - 2: $B_1 \leftarrow g_1^{b_1} g_2^{b_2} \bmod N_j$
 - 3: $B_2 \leftarrow g_1^{b_2} g_2^{b_1} \bmod N_j$
 - 4: $c \leftarrow m + e_1^{b_1} e_2^{b_2} \bmod N_j$
 - 5: **return** (B_1, B_2, c)
-

Algorithm 6 KAZ-KEM Decapsulation Algorithm

Require: Private key (a_1, a_2) , ciphertext (B_1, B_2, c)

Ensure: Plaintext m

- 1: $y \leftarrow c - B_1^{a_1} B_2^{a_2} \bmod N_j$
 - 2: **return** y
-

KAZ-SIGN v2.0. The digital signature scheme consists of three procedures. Algorithm 7 generates the verification key V from secrets $s \in \mathbb{Z}_{o_1}$ and $t \in \mathbb{Z}_{o_2}$. Algorithm 8 signs a message by sampling fresh $e_1 \in \mathbb{Z}_{o_1}$ and $e_2 \in \mathbb{Z}_{o_2}$, computing S_1 , and then deriving S_2 and S_3 so that $sS_1 + e_1S_2 \equiv h$ and $tS_1 + e_2S_2 + S_3 \equiv h \pmod{\varphi(N_j)}$. Algorithm 9 checks the signature by verifying the relation $V^{S_1} S_1^{S_2} g_2^{S_3} \equiv (g_1 g_2)^h \pmod{N_j}$. Correctness follows because $Y_1 = V^{S_1} S_1^{S_2} g_2^{S_3} = g_1^{sS_1 + e_1S_2} g_2^{tS_1 + e_2S_2 + S_3} \equiv (g_1 g_2)^h = Y_2 \pmod{N_j}$.

Algorithm 7 KAZ-SIGN v2.0 Key Generation**Require:** System parameters $(g_1, g_2, o_1, o_2, N_j)$ **Ensure:** Verification key V , signing key (s, t)

- 1: Choose random $s \in \mathbb{Z}_{o_1}$ and $t \in \mathbb{Z}_{o_2}$
- 2: $V \leftarrow g_1^s g_2^t \bmod N_j$
- 3: **return** $V, (s, t)$

Algorithm 8 KAZ-SIGN v2.0 Signing**Require:** System parameters $(g_1, g_2, o_1, o_2, N_j)$, signing key (s, t) , hash function H , message m **Ensure:** Signature $\sigma = (S_1, S_2, S_3)$

- 1: $h \leftarrow H(m)$
- 2: Choose random $e_1 \in \mathbb{Z}_{o_1}$ and $e_2 \in \mathbb{Z}_{o_2}$
- 3: $S_1 \leftarrow g_1^{e_1} g_2^{e_2} \bmod N_j$
- 4: $S_2 \leftarrow (h - s \cdot S_1) \cdot e_1^{-1} \bmod \varphi(N_j)$
- 5: $S_3 \leftarrow h - t \cdot S_1 - e_2 \cdot S_2 \bmod \varphi(N_j)$
- 6: **return** (S_1, S_2, S_3)

Algorithm 9 KAZ-SIGN v2.0 Verification**Require:** System parameters $(g_1, g_2, o_1, o_2, N_j)$, verification key V , hash function H , message m , signature (S_1, S_2, S_3) **Ensure:** Accept or Reject

- 1: $h \leftarrow H(m)$
- 2: $Y_1 \leftarrow V^{S_1} \cdot S_1^{S_2} \cdot g_2^{S_3} \bmod N_j$
- 3: $Y_2 \leftarrow (g_1 \cdot g_2)^h \bmod N_j$
- 4: **if** $Y_1 = Y_2$ **then**
- 5: **return** Accept
- 6: **else**
- 7: **return** Reject
- 8: **end if**

Table 1 summarizes the recommended sizes. Since N_j is a primorial of small primes, it is extremely smooth and can be factored quickly, which enables a Pohlig–Hellman style attack by reducing DLPs in $\mathbb{Z}_{N_j}^*$ to DLPs modulo each small prime p_i . Modulo a given p_i , even though two public bases g_1 and g_2 appear, we can guess one exponent and solve a single-base DLP for the other to obtain candidate secrets. For KAZ-KA and KAZ-KEM, the two public-key equations allow strong pruning across moduli. For KAZ-SIGN v2.0, only one equation is available, which leaves a large candidate space. This motivates our HNP attack derived from the signature equations to recover the signing key.

Table 1. Shared parameter sizes for KAZ-KA, KAZ-KEM, and KAZ-SIGN v2.0.

Security Level	j	p_{j+1}	$\text{len}(N_j)$	$\text{len}(\varphi(N_j))$
128-bit	65	317	432	430
192-bit	96	509	702	700
256-bit	122	677	942	940

KAZ-SIGN v1.6.4. This variant does not follow the primorial-based dual-generator structure. Its public parameters are $(k, q, \rho, Q, R, G_0, G_1, A, L_{G_1\rho}, L_{G_1qQ\rho})$, where $Q = \prod p_i$ is the product of consecutive odd primes from a public list, q is a prime with $\text{len}(q) \approx \text{len}(Q)$, and $\rho = 2\rho_0 + 1$ with ρ_0 prime. The specification defines Q as the product of k_0 consecutive odd primes with $k_0 < j$; the k_0 values in Table 2 are computed from the odd primes in the reference implementation. For the 128- and 256-bit parameter sets, the hardcoded Q additionally includes a factor 2^2 , which is not fully aligned with the odd-prime-only definition. The list is the same source used to define N_j in the other three schemes, so Q is a primorial factor drawn from that list, but KAZ-SIGN v1.6.4 itself does not define or depend on N_j . The composite $G_0 = 2^{10} \prod p_i^{e_i}$ is formed from primes in that list with small exponents $e_i \in \mathbb{Z}_{11}$; R is a prime in $\mathbb{Z}_{G_0}^*$ and $G_1 = \text{ord}_{G_0}(R)$. Let A be the product of the prime-power factors of G_1 , whose largest prime factor is about 2^4 , and define $G_{1A} = G_1\rho/A$, with $L_{G_1\rho} = \text{len}(G_1\rho)$ and $L_{G_1qQ\rho} = \text{len}(G_1qQ\rho)$.

Key generation samples a secret α and outputs $V_1 \equiv \alpha \pmod{G_1\rho}$, $V_2 \equiv Q \cdot \alpha^{\varphi(Q)^b} \pmod{qQ}$, and $SK \equiv \alpha^{\varphi(Q)^b} \pmod{G_1qQ\rho}$ (Algorithm 10). The verification algorithm checks a main relation of the form $R^{S_1} \equiv R^T \pmod{G_0}$, together with multiple filter conditions and an additional congruence on G_{1A} . We fully break this version. We show that its public verification structure sharply reduces the search space and enables public-key signature forgery. We do not recover the private key.

Algorithm 10: KAZ-SIGN v1.6.4 Key Generation

Require: System parameters $(k, q, \rho, Q, R, G_0, G_1, A, L_{G_1\rho})$

Ensure: Verification key (V_1, V_2) , signing key SK

- 1: Sample a 32-bit prime a and a random 32-bit integer ω_1
 - 2: $\rho_0 \leftarrow (\rho - 1)/2$
 - 3: $b \leftarrow a^{\varphi(G_1)(\rho_0-1)} \pmod{(\omega_1\varphi(G_1)(\rho-1))}$
 - 4: $G_{1A} \leftarrow G_1\rho/A$
 - 5: **repeat**
 - 6: Choose even α with bit-length $k + L_{G_1\rho}$
 - 7: $V_1 \leftarrow \alpha \pmod{G_1\rho}$
 - 8: $V_2 \leftarrow Q \cdot \alpha^{\varphi(Q)^b} \pmod{qQ}$
 - 9: $SK \leftarrow \alpha^{\varphi(Q)^b} \pmod{G_1qQ\rho}$
 - 10: **until** $\text{gcd}(V_1, G_{1A}) = 1$ and $SK \not\equiv 0 \pmod{G_1qQ\rho}$
 - 11: **return** $(V_1, V_2), SK$
-

Algorithm 11 gives the signing procedure. Given (V_1, V_2) and SK , the signer initializes $h = H(m)$ and $S_2 = 0$, then samples 32-bit primes r_1, r_2 and 32-bit integers ω_2, ω_3 to derive $\beta_i \equiv r_i^{\varphi(G_1)(\rho_0-1)} \pmod{(\omega_{i+1}\varphi(G_1)(\rho-1))}$. The signature core is $S_1 \equiv SK \cdot (h^{\varphi(qQ)\beta_1} + h^{\varphi(qQ)\beta_2}) \pmod{G_1qQ\rho}$. It then computes $Y_1 \equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1Q\rho}$ and $SF_1 = \text{CRT}([V_2/Q, Y_1], [q, G_1Q\rho])$, and repeats with $h \leftarrow h + 1$, $S_2 \leftarrow S_2 + 1$ until $\text{len}(S_1) = L_{G_1qQ\rho}$ and $(S_1 \bmod G_1qQ\rho) \neq SF_1$.

Algorithm 11: KAZ-SIGN v1.6.4 Signing

Require: System parameters $(q, \rho, Q, G_1, L_{G_1qQ\rho})$, public key (V_1, V_2) , signing key SK , message m

Ensure: Signature (S_1, S_2)

```

1:  $h \leftarrow H(m)$ ,  $S_2 \leftarrow 0$ ,  $\rho_0 \leftarrow (\rho - 1)/2$ 
2:  $valid \leftarrow 0$ 
3: while  $valid = 0$  do
4:   Sample a 32-bit prime  $r_1$  and a random 32-bit integer  $\omega_2$ 
5:    $\beta_1 \leftarrow r_1^{\varphi(G_1)(\rho_0-1)} \pmod{(\omega_2\varphi(G_1)(\rho-1))}$ 
6:   Sample a 32-bit prime  $r_2$  and a random 32-bit integer  $\omega_3$ 
7:    $\beta_2 \leftarrow r_2^{\varphi(G_1)(\rho_0-1)} \pmod{(\omega_3\varphi(G_1)(\rho-1))}$ 
8:    $S_1 \leftarrow SK \cdot (h^{\varphi(qQ)\beta_1} + h^{\varphi(qQ)\beta_2}) \pmod{G_1qQ\rho}$ 
9:    $Y_1 \leftarrow V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1Q\rho}$ 
10:   $SF_1 \leftarrow \text{CRT}([V_2/Q, Y_1], [q, G_1Q\rho])$ 
11:   $w_1 \leftarrow \text{len}(S_1) - L_{G_1qQ\rho}$ 
12:   $w_2 \leftarrow (S_1 \bmod G_1qQ\rho) - SF_1$ 
13:  if  $w_1 \neq 0$  or  $w_2 = 0$  then
14:     $h \leftarrow h + 1$ ,  $S_2 \leftarrow S_2 + 1$ 
15:  else
16:     $valid \leftarrow 1$ 
17:  end if
18: end while
19: return  $(S_1, S_2)$ 

```

Algorithm 12 gives the verification procedure. The verifier recomputes $h = H(m) + S_2$, Y_1 , and SF_1 as above, and also computes $Y_2 \equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1\rho}$ and $SF_2 = \text{CRT}([V_2/Q, Y_2], [qQ/e, G_1\rho])$ with $e = \gcd(Q, G_1)$. It then applies the five filters: (i) $S_1 < G_1qQ\rho$, (ii) $\text{len}(S_1) = L_{G_1qQ\rho}$, (iii) $S_1 \neq SF_1 \pmod{G_1qQ\rho}$, (iv) $S_1 \not\equiv SF_2 \pmod{G_1qQ\rho/e}$, and (v) $QS_1 \equiv 2V_2 \pmod{qQ}$. The main verification checks $R^{S_1} \equiv R^T \pmod{G_0}$ with $T \equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1}$, followed by the additional congruence $S_1 \cdot (V_1^{\varphi(Q)})^{-1} \equiv 2h^{\varphi(qQ)} \pmod{G_{1A}}$.

Algorithm 12: KAZ-SIGN v1.6.4 Verification

Require: System parameters $(q, \rho, Q, R, G_0, G_1, A, L_{G_1qQ\rho})$, public key (V_1, V_2) , message m , signature (S_1, S_2)

Ensure: Accept or Reject

```

1:  $h \leftarrow H(m) + S_2$ 
2:  $Y_1 \leftarrow V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \bmod G_1Q\rho$ 
3:  $SF_1 \leftarrow \text{CRT}([V_2/Q, Y_1], [q, G_1Q\rho])$ 
4:  $e \leftarrow \gcd(Q, G_1)$ 
5:  $Y_2 \leftarrow V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \bmod G_1\rho$ 
6:  $SF_2 \leftarrow \text{CRT}([V_2/Q, Y_2], [qQ/e, G_1\rho])$ 
7: if  $(S_1 \bmod G_1qQ\rho) - S_1 \neq 0$  then
8:   return Reject
9: end if
10: if  $\text{len}(S_1) \neq L_{G_1qQ\rho}$  then
11:   return Reject
12: end if
13: if  $(S_1 \bmod G_1qQ\rho) - SF_1 = 0$  then
14:   return Reject
15: end if
16: if  $(S_1 \bmod (G_1qQ\rho/e)) - SF_2 = 0$  then
17:   return Reject
18: end if
19: if  $QS_1 \not\equiv 2V_2 \pmod{qQ}$  then
20:   return Reject
21: end if
22:  $y_1 \leftarrow R^{S_1} \bmod G_0$ 
23:  $T \leftarrow V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \bmod G_1$ 
24:  $y_2 \leftarrow R^T \bmod G_0$ 
25: if  $y_1 \neq y_2$  then
26:   return Reject
27: end if
28:  $G_{1A} \leftarrow G_1\rho/A$ 
29:  $y_3 \leftarrow S_1 \cdot (V_1^{\varphi(Q)})^{-1} \bmod G_{1A}$ 
30:  $y_4 \leftarrow 2h^{\varphi(qQ)} \bmod G_{1A}$ 
31: if  $y_3 \neq y_4$  then
32:   return Reject
33: end if
34: return Accept

```

Table 2. KAZ-SIGN v1.6.4 parameter sizes ($G_{1A} = G_{1\rho}/A$).

Security	k_0	$\text{len}(G_1)$	$\text{len}(q)$	$\text{len}(Q)$	$\text{len}(\rho)$	$\text{len}(G_{1A})$
128-bit	19	79	88	90	129	179
192-bit	29	79	153	154	193	243
256-bit	37	79	221	213	257	307

5 Attacks on KAZ Schemes

We present practical attacks on all four KAZ schemes. For KAZ-KA and KAZ-KEM, we exploit the smooth group order to solve discrete logarithms modulo each small prime factor independently. For KAZ-SIGN v2.0, we formulate the signature equations as a HNP and solve it via lattice reduction. For KAZ-SIGN v1.6.4, we do not recover the signing key, but we can forge signatures from the public key. All experiments are conducted on a MacBook Air with an Apple M3 chip, 8 cores, and 24GB RAM.

5.1 Key Recovery Attack on KAZ-KA and KAZ-KEM

Pohlig–Hellman is effective when the group order is smooth because it reduces a DLP in a group of order n to DLPs in subgroups of order $q_i^{e_i}$ and recombines the solutions via the CRT. When all q_i are small, these subproblems are cheap, and the overall DLP becomes practical.

Our key recovery attack on KAZ-KA and KAZ-KEM recovers the private key (a_1, a_2) from the public key (A_1, A_2) in polynomial time. The attack exploits the fact that N_j is a primorial, allowing us to work modulo each small prime factor independently.

The recommended parameters use $j = 65, 96, 122$ consecutive odd primes for 128-, 192-, and 256-bit security levels respectively. The modulus $N_j = 3 \cdot 5 \cdot 7 \cdots p_{j+1}$ has the largest prime factor $p_{j+1} = 317, 509, 677$ for these security levels.

We start from the public key equations:

$$A_1 \equiv g_1^{a_1} g_2^{2a_2} \pmod{N_j}, \quad (1)$$

$$A_2 \equiv g_1^{a_2} g_2^{a_1} \pmod{N_j}. \quad (2)$$

Since $N_j = \prod_{i=2}^{j+1} p_i$, by the CRT we can work independently modulo each prime factor p_i . For each p_i , equations (1) and (2) reduce to:

$$A_1 \equiv g_1^{a_1} g_2^{2a_2} \pmod{p_i}, \quad (3)$$

$$A_2 \equiv g_1^{a_2} g_2^{a_1} \pmod{p_i}. \quad (4)$$

Step 1: Enumerate and Solve DLP. For each prime p dividing N_j , let $\omega_1 = \text{ord}_p(g_1)$ and $\omega_2 = \text{ord}_p(g_2)$ be the multiplicative orders of g_1 and g_2 in $\mathbb{Z}_p^*$. For each candidate value $u \in [0, \omega_1)$ representing a guess for $a_1 \bmod \omega_1$, we compute:

$$T_1 \equiv A_1 \cdot g_1^{-u} \equiv g_2^{2a_2} \pmod{p}.$$

We then solve for $\log_{g_2} T_1$ in $\mathbb{Z}_p^*$ to obtain $2a_2 \bmod \omega_2$, and subsequently $a_2 \bmod \omega_2$ (handling the factor of 2 appropriately).

For each candidate pair (u, v) where $u = a_1 \bmod \omega_1$ and $v = a_2 \bmod \omega_2$, we verify consistency with equation (4) by checking:

$$A_2 \cdot g_2^{-v} \equiv g_1^u \pmod{p}.$$

Valid candidates that satisfy both equations are retained.

Step 2: Combine via CRT. Let $L_p = \text{lcm}(\omega_1, \omega_2)$ for each prime p . For each prime, we obtain a set of candidate solutions $(a_1 \bmod L_p, a_2 \bmod L_p)$. We use the CRT to merge solutions across all primes. The final candidates are verified against the original public key (A_1, A_2) to recover the correct private key (a_1, a_2) .

We can estimate the time complexity per prime factor. The enumeration step tries $\omega_1 \leq p - 1$ candidates for $a_1 \bmod \omega_1$. For each candidate we solve $\log_{g_2} T_1$ in $\mathbb{Z}_p^*$ using baby-step giant-step [25], which costs $O(\sqrt{\omega_2})$ time and space with $\omega_2 \leq p - 1$. Since $\omega_1, \omega_2 \leq p - 1$, the per-prime complexity is $O(\omega_1 \cdot \sqrt{\omega_2}) = O(p \cdot \sqrt{p}) = O(p^{3/2})$. Combining candidates across primes via the CRT adds only linear overhead relative to the number of candidates.

The total complexity over all j primes is:

$$\sum_{i=2}^{j+1} O(p_i^{3/2}) = O\left(\sum_{i=2}^{j+1} p_i^{3/2}\right).$$

By the prime number theorem, $p_i \approx i \ln i$, so the sum is dominated by the largest terms and is approximately $O(j \cdot p_{j+1}^{3/2})$.

In practice, the per-prime DLPs are tiny. Each p is at most a few hundred, so we precompute discrete logarithm tables:

$$\begin{aligned} \text{dlog}_1 : g_1^k \bmod p &\mapsto k, & \text{for } k \in [0, \omega_1), \\ \text{dlog}_2 : g_2^k \bmod p &\mapsto k, & \text{for } k \in [0, \omega_2). \end{aligned}$$

Each table requires $O(p)$ time and space, which is negligible at these sizes. With precomputed tables, each DLP lookup takes $O(1)$ time, reducing the per-prime complexity to $O(p)$ and the total complexity to $O(j \cdot p_{j+1})$. The tables are computed once per prime and reused across all key instances, so the online cost is dominated by simple modular checks.

Algorithm 13 summarizes the attack procedure. For each prime factor p of N_j , we build discrete logarithm tables for g_1 and g_2 , then enumerate candidate values u for $a_1 \bmod \omega_1$. For each candidate u , we compute $T_1 = A_1/g_1^u$ and look up its discrete logarithm in the g_2 -table. The public key equation $A_1 = g_1^{a_1} g_2^{2a_2}$ implies that when $u = a_1$, the exponent of g_2 in T_1 must be even. We use this constraint to filter invalid candidates. We then verify consistency with the second public key equation $A_2 = g_1^{a_2} g_2^{a_1}$. Valid local solutions are combined via CRT across all primes to recover the full private key.

Algorithm 13 Key Recovery Attack on KAZ-KA/KEM**Require:** Public key (A_1, A_2) , modulus N_j , generators g_1, g_2 **Ensure:** Private key (a_1, a_2)

```

1:  $\mathcal{P} \leftarrow \{p : p \mid N_j, p \text{ prime}\}$  {Factor  $N_j$ }
2:  $\mathcal{C} \leftarrow \emptyset$  {Candidate set}
3: for each prime  $p \in \mathcal{P}$  do
4:    $\omega_1 \leftarrow \text{ord}_p(g_1)$  {Order of  $g_1$  in  $\mathbb{Z}_p^*$ }
5:    $\omega_2 \leftarrow \text{ord}_p(g_2)$  {Order of  $g_2$  in  $\mathbb{Z}_p^*$ }
6:    $L_p \leftarrow \text{lcm}(\omega_1, \omega_2)$ 
7:   Build table  $\text{dlog}_1$ : for  $k = 0, \dots, \omega_1 - 1$ , set  $\text{dlog}_1[g_1^k \bmod p] \leftarrow k$ 
8:   Build table  $\text{dlog}_2$ : for  $k = 0, \dots, \omega_2 - 1$ , set  $\text{dlog}_2[g_2^k \bmod p] \leftarrow k$ 
9:    $\mathcal{S}_p \leftarrow \emptyset$  {Local solutions mod  $p$ }
10:  for  $u = 0$  to  $\omega_1 - 1$  do
11:     $T_1 \leftarrow A_1 \cdot g_1^{-u} \bmod p$ 
12:    if  $T_1 \notin \text{dlog}_2$  then
13:      continue
14:    end if
15:     $\text{exp} \leftarrow \text{dlog}_2[T_1]$ 
16:    if  $\text{exp} \bmod 2 \neq 0$  then
17:      continue {Need 2 | exp for  $v$ }
18:    end if
19:     $v \leftarrow (\text{exp} / 2) \bmod \omega_2$ 
20:     $T_2 \leftarrow A_2 \cdot g_1^{-v} \cdot g_2^{-u} \bmod p$ 
21:    if  $T_2 \equiv 1 \pmod{p}$  then
22:       $\mathcal{S}_p \leftarrow \mathcal{S}_p \cup \{(u \bmod L_p, v \bmod L_p)\}$ 
23:    end if
24:  end for
25:   $\mathcal{C} \leftarrow \text{CRT-Merge}(\mathcal{C}, \mathcal{S}_p)$  {Combine via CRT}
26: end for
27: for each  $(a_1, a_2) \in \mathcal{C}$  do
28:   if  $g_1^{a_1} g_2^{2a_2} \equiv A_1 \pmod{N_j}$  then
29:    if  $g_1^{a_2} g_2^{a_1} \equiv A_2 \pmod{N_j}$  then
30:      return  $(a_1, a_2)$ 
31:    end if
32:  end if
33: end for

```

Table 3. Attack results on KAZ-KA and KAZ-KEM

Security Level	Num of Primes	$\text{len}(a_1), \text{len}(a_2)$	Time (s)	Success Rate
128-bit	65	128	0.019	100%
192-bit	96	192	0.032	100%
256-bit	122	256	0.058	100%

For the recommended parameters with $p_{j+1} < 700$ and $j \leq 122$, the attack completes in $O(10^5)$ operations. We implement the attack in SageMath and test on all three security levels. Table 3 summarizes the results, with timing averaged over 100 runs. Each run uses a freshly generated random key pair. The attack successfully recovers the private key in under 0.1 seconds for all parameter sets.

5.2 Lattice Attack on KAZ-SIGN v2.0

For KAZ-SIGN v2.0, the public key $V \equiv g_1^s g_2^t \pmod{N_j}$ provides only a single equation in the two unknown secrets (s, t) . A direct application of the table-based approach from Section 5.1 would require enumerating all possible values of one secret, leading to exponential complexity. Instead, we develop a lattice-based attack that exploits the algebraic structure of the signature scheme, requiring only two signatures. The two signatures can be on different messages, since the hash values h are public and computable.

We begin by recalling the signature equations, which make explicit how the short ephemeral values (e_1, e_2) interact with the long-term secrets (s, t) . A signature $\sigma = (S_1, S_2, S_3)$ on message m with hash $h = H(m)$ satisfies:

$$S_1 \equiv g_1^{e_1} g_2^{e_2} \pmod{N_j}, \quad (5)$$

$$S_2 \equiv (h - s \cdot S_1) \cdot e_1^{-1} \pmod{\varphi(N_j)}, \quad (6)$$

$$S_3 \equiv h - t \cdot S_1 - e_2 \cdot S_2 \pmod{\varphi(N_j)}, \quad (7)$$

where (e_1, e_2) are ephemeral values chosen during signing.

Rearranging equation (6):

$$s \cdot S_1 + e_1 \cdot S_2 \equiv h \pmod{\varphi(N_j)}. \quad (8)$$

Similarly, from equation (7):

$$t \cdot S_1 + e_2 \cdot S_2 \equiv h - S_3 \pmod{\varphi(N_j)}. \quad (9)$$

To expose the hidden-number structure, we normalize each equation because $S_2^{(i)}$ may not be invertible modulo $\varphi(N_j)$. We remove the non-invertible factor of $S_2^{(i)}$ by letting $g^{(i)} = \gcd(S_2^{(i)}, \varphi(N_j))$ and defining $\varphi_i = \varphi(N_j)/g^{(i)}$ and $\tilde{S}_2^{(i)} = S_2^{(i)}/g^{(i)}$. Then $\tilde{S}_2^{(i)}$ is invertible modulo φ_i , and dividing equation (8) by $S_2^{(i)}$ yields:

$$e_1^{(i)} \equiv b_i - s \cdot a_i \pmod{\varphi_i}, \quad (10)$$

where $a_i = S_1^{(i)} \cdot (\tilde{S}_2^{(i)})^{-1} \pmod{\varphi_i}$ and $b_i = h_i \cdot (\tilde{S}_2^{(i)})^{-1} \pmod{\varphi_i}$.

With two signatures we obtain two congruences in the same secret s with small offsets $e_1^{(i)}$. Concretely, equation (10) gives:

$$e_1^{(1)} \equiv b_1 - s \cdot a_1 \pmod{\varphi_1},$$

$$e_1^{(2)} \equiv b_2 - s \cdot a_2 \pmod{\varphi_2}.$$

Table 4. Attack results on KAZ-SIGN v2.0 (two signatures)

Security Level	$\text{len}(\varphi(N_j))$	$\text{len}(s), \text{len}(t)$	Time (s)	Success Rate
128-bit	430	128	0.002	100%
192-bit	700	192	0.001	100%
256-bit	940	256	0.001	100%

We construct the lattice $\mathcal{L}_1$ spanned by the rows of:

$$\mathbf{B}_1 = \begin{pmatrix} \varphi_1 & 0 & 0 & 0 \\ 0 & \varphi_2 & 0 & 0 \\ -a_1 & -a_2 & 1 & 0 \\ b_1 & b_2 & 0 & 1 \end{pmatrix}.$$

By construction, the vector $\mathbf{v} = (e_1^{(1)}, e_1^{(2)}, s, 1)$ lies in $\mathcal{L}_1$. Since $e_1^{(i)}$ are approximately k -bit integers while $\varphi_i \approx 2^{3k}$, this vector is significantly shorter than a random lattice vector. Using the Gaussian heuristic from the preliminaries, the expected scale of $\lambda_1(\mathcal{L}_1)$ is $\text{gh}(\mathcal{L}_1) \approx (\det(\mathcal{L}_1))^{1/4} \sqrt{4/(2\pi e)}$ with $\det(\mathcal{L}_1) = \varphi_1 \varphi_2$, so $\text{gh}(\mathcal{L}_1)$ is on the order of $2^{3k/2}$ whereas $\|\mathbf{v}\|$ is on the order of 2^k . This gap suggests that LLL should expose a short vector that reveals s . Applying LLL to $\mathbf{B}_1$, we search for a row $\mathbf{r} = (r_1, r_2, r_3, r_4)$ with $|r_4| = 1$; the signing key is $s = r_3/r_4$.

We recover t in the same way by using the second signing equation. From equation (9), we have $e_2^{(i)} \equiv c_i - t \cdot a_i \pmod{\varphi_i}$, where $c_i = (h_i - S_3^{(i)}) \cdot (\tilde{S}_2^{(i)})^{-1} \pmod{\varphi_i}$. This yields another pair of congruences with small errors, and we construct an analogous lattice $\mathcal{L}_2$ spanned by the rows of:

$$\mathbf{B}_2 = \begin{pmatrix} \varphi_1 & 0 & 0 & 0 \\ 0 & \varphi_2 & 0 & 0 \\ -a_1 & -a_2 & 1 & 0 \\ c_1 & c_2 & 0 & 1 \end{pmatrix},$$

and apply LLL to recover t .

Algorithm 14 summarizes the complete attack procedure. For each signature, we first preprocess it by computing the reduced modulus φ_i and the normalized coefficients a_i, b_i, c_i . We then construct a 4-dimensional lattice $\mathcal{L}_1$ whose short vector encodes the secret s along with the small ephemeral values, and apply LLL to find this short vector. The target vector $(e_1^{(1)}, e_1^{(2)}, s, 1)$ is much shorter than a random lattice vector due to the small size of the ephemeral values $e_1^{(i)}$, which makes it recoverable by LLL. Finally, we construct an analogous lattice $\mathcal{L}_2$ to recover the second signing key t .

Algorithm 14 Lattice Attack on KAZ-SIGN v2.0**Require:** Two signatures $\sigma^{(1)}, \sigma^{(2)}$ on messages with hashes h_1, h_2 **Require:** Public parameters $N_j, \varphi(N_j)$ **Ensure:** Private key (s, t)

```

1: {Step 1: Preprocess signatures}
2: for  $i = 1, 2$  do
3:   Parse  $\sigma^{(i)} = (S_1^{(i)}, S_2^{(i)}, S_3^{(i)})$ 
4:    $g_i \leftarrow \gcd(S_2^{(i)}, \varphi(N_j))$  {Remove non-invertible factor}
5:    $\varphi_i \leftarrow \varphi(N_j)/g_i$ 
6:    $\tilde{S}_2^{(i)} \leftarrow S_2^{(i)}/g_i$ 
7:    $a_i \leftarrow S_1^{(i)} \cdot (\tilde{S}_2^{(i)})^{-1} \bmod \varphi_i$ 
8:    $b_i \leftarrow h_i \cdot (\tilde{S}_2^{(i)})^{-1} \bmod \varphi_i$ 
9:    $c_i \leftarrow (h_i - S_3^{(i)}) \cdot (\tilde{S}_2^{(i)})^{-1} \bmod \varphi_i$ 
10: end for
11:
12: {Step 2: Recover  $s$  via lattice reduction}
13: Construct lattice basis  $\mathbf{B}_1 \leftarrow \begin{pmatrix} \varphi_1 & 0 & 0 & 0 \\ 0 & \varphi_2 & 0 & 0 \\ -a_1 & -a_2 & 1 & 0 \\ b_1 & b_2 & 0 & 1 \end{pmatrix}$ 
14:  $\mathbf{L}_1 \leftarrow \text{LLL}(\mathbf{B}_1)$ 
15: for each row  $\mathbf{r} = (r_1, r_2, r_3, r_4)$  in  $\mathbf{L}_1$  do
16:   if  $|r_4| = 1$  then
17:      $s \leftarrow r_3/r_4$  {Short vector reveals  $s$  ( $r_4 = \pm 1$ )}
18:   end if
19: end for
20:
21: {Step 3: Recover  $t$  via lattice reduction}
22: Construct lattice basis  $\mathbf{B}_2 \leftarrow \begin{pmatrix} \varphi_1 & 0 & 0 & 0 \\ 0 & \varphi_2 & 0 & 0 \\ -a_1 & -a_2 & 1 & 0 \\ c_1 & c_2 & 0 & 1 \end{pmatrix}$ 
23:  $\mathbf{L}_2 \leftarrow \text{LLL}(\mathbf{B}_2)$ 
24: for each row  $\mathbf{r} = (r_1, r_2, r_3, r_4)$  in  $\mathbf{L}_2$  do
25:   if  $|r_4| = 1$  then
26:      $t \leftarrow r_3/r_4$  {Short vector reveals  $t$  ( $r_4 = \pm 1$ )}
27:   end if
28: end for
29:
30: return  $(s, t)$ 

```

Computing $(a_i, b_i, c_i, \varphi_i)$ involves modular arithmetic on roughly $3k$ -bit integers, after which forming $\mathbf{B}_1$ and $\mathbf{B}_2$ is negligible. In practice, the runtime is dominated by a handful of big-integer operations, and 4-dimensional LLL is effectively instantaneous. We implement the attack in SageMath and evaluate all three security levels; Table 4 reports averages over 100 runs with fresh keys

and two random-message signatures per run. With two signatures, LLL recovers the private key with 100% success in our experiments.

5.3 Signature Forgery Attack on KAZ-SIGN v1.6.4

We analyze the public-key relations implied by Algorithms 10–12, and show that a forger can construct a valid signature (S_1, S_2) for any message m *without access to any prior signatures*. The signer uses S_2 to iterate $h = H(m) + S_2$ until the range and exclusion filters pass. In our forgery, we search S_2 over 16-bit values and compute $h = H(m) + S_2$. For each S_2 we compute a candidate S_1 from the public congruences and then check the remaining filters.

Algorithm 15 summarizes the forgery procedure. The verifier checks the main congruence $R^{S_1} \equiv R^T \pmod{G_0}$ with $T \equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1}$. Since $\text{ord}_{G_0}(R) = G_1$, this is equivalent to

$$S_1 \equiv T \pmod{G_1}.$$

Filter (v) enforces $QS_1 \equiv 2V_2 \pmod{qQ}$, and the public key relation $V_2 \equiv Q \cdot SK \pmod{qQ}$ makes $SK \pmod{q}$ public, yielding

$$S_1 \equiv 2 \cdot SK \pmod{q}.$$

Finally, the additional congruence on G_{1A} gives

$$S_1 \equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_{1A}}.$$

Combining these constraints via CRT yields a unique residue class

$$S_1 \equiv s_1 \pmod{\text{lcm}(G_1, q, G_{1A})}.$$

Because $G_{1A} = G_1\rho/A$ with q and ρ distinct primes, the modulus simplifies to

$$\text{lcm}(G_1, q, G_{1A}) = G_1q\rho.$$

Write $S_1 = s_1 + k \cdot G_1q\rho$ for some integer k . The range filters (i)–(ii) require $S_1 \in [2^{L_{G_1qQ\rho}-1}, G_1qQ\rho - 1]$, so k must satisfy

$$\left\lceil \frac{2^{L_{G_1qQ\rho}-1} - s_1}{G_1q\rho} \right\rceil \leq k \leq \left\lfloor \frac{G_1qQ\rho - 1 - s_1}{G_1q\rho} \right\rfloor.$$

Thus, k ranges over a contiguous interval on the order of Q , and once s_1 is fixed, there are roughly Q admissible candidates for S_1 .

Let

$$\begin{aligned} Y_1 &\equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1Q\rho}, \\ Y_2 &\equiv V_1^{\varphi(Q)} \cdot 2h^{\varphi(qQ)} \pmod{G_1\rho}. \end{aligned}$$

The remaining verification steps exclude the two CRT residues

$$SF_1 = \text{CRT}([V_2/Q, Y_1], [q, G_1 Q \rho]),$$

$$SF_2 = \text{CRT}([V_2/Q, Y_2], [qQ/e, G_1 \rho]),$$

where $e = \gcd(Q, G_1)$. We analyze these two exclusions in turn.

Algorithm 15 Signature Forgery Attack on KAZ-SIGN v1.6.4

Require: Public key (V_1, V_2) , message m , public parameters

Ensure: Forged signature (S_1, S_2)

```

1:  $h \leftarrow H(m)$ 
2:  $S_1^{(q)} \leftarrow 2V_2Q^{-1} \bmod q$ 
3:  $G_{1A} \leftarrow G_1\rho/A$ 
4:  $g \leftarrow \gcd(G_1, G_{1A}), L \leftarrow \text{lcm}(G_1, G_{1A})$ 
5: for  $S_2 = 0$  to  $2^{16} - 1$  do
6:    $h' \leftarrow h + S_2$ 
7:    $T_1 \leftarrow V_1^{\varphi(Q)} \cdot 2h'^{\varphi(qQ)} \bmod G_1$ 
8:    $T_2 \leftarrow V_1^{\varphi(Q)} \cdot 2h'^{\varphi(qQ)} \bmod G_{1A}$ 
9:   if  $T_1 \not\equiv T_2 \pmod{g}$  then
10:    continue
11:   end if
12:    $T \leftarrow \text{CRT}([T_1, T_2], [G_1, G_{1A}])$ 
13:   if  $T \not\equiv S_1^{(q)} \pmod{\gcd(L, q)}$  then
14:    continue
15:   end if
16:    $S_1^{(0)} \leftarrow \text{CRT}([T, S_1^{(q)}], [L, q])$ 
17:    $M \leftarrow \text{lcm}(L, q)$ 
18:   Compute  $k_{\min}$  from the range  $2^{L_{G_1qQ\rho}-1} \leq S_1 \leq G_1qQ\rho - 1$ 
19:   for  $k = k_{\min}$  to  $\min(k_{\max}, k_{\min} + 10^5)$  do
20:      $S_1 \leftarrow S_1^{(0)} + kM$ 
21:     if  $S_1$  is out of range or has wrong bit length then
22:       continue
23:     end if
24:     Compute  $SF_1$  and  $SF_2$  using  $h'$ 
25:     if  $S_1$  violates Filter (iii) or Filter (iv) then
26:       continue
27:     end if
28:     return  $(S_1, S_2)$ 
29:   end for
30: end for
31: return failure

```

Filter (iii). If $S_1 \equiv SF_1 \pmod{G_1qQ\rho}$, then simultaneously $S_1 \equiv V_2/Q \pmod{q}$ and $S_1 \equiv Y_1 \pmod{G_1Q\rho}$. However, Filter (v) already fixes $S_1 \equiv 2V_2/Q \pmod{q}$. Since q is an odd prime, the two residues are incompatible unless $V_2/Q \equiv 0 \pmod{q}$, which would require $\alpha \equiv 0 \pmod{q}$ and occurs with proba-

bility $1/q$ under key generation. In this exceptional case, writing $S_1 = s_1 + k \cdot G_1 q \rho$ and reducing modulo $G_1 Q \rho$ yields

$$k \equiv (Y_1 - s_1) \cdot (G_1 \rho)^{-1} \pmod{Q},$$

so at most one value of k is excluded in the admissible interval.

Filter (iv). If $S_1 \equiv SF_2 \pmod{G_1 q Q \rho / e}$, then $S_1 \equiv V_2 / Q \pmod{q}$ and $S_1 \equiv Y_2 \pmod{G_1 \rho}$. The same mod- q mismatch with Filter (v) makes this condition impossible except in the same negligible case $V_2 / Q \equiv 0 \pmod{q}$. In that case, since $G_1 q \rho = q \cdot (G_1 \rho)$ and $\gcd(G_1 \rho, Q/e) = 1$, reducing the congruence modulo qQ/e gives

$$k \cdot G_1 \rho \equiv (SF_2 - s_1) / q \pmod{Q/e},$$

which fixes k modulo Q/e and thus excludes at most one candidate in the admissible range.

Overall, for keys with $V_2 / Q \not\equiv 0 \pmod{q}$ (which occurs with probability $1 - 1/q$ under key generation), Filters (iii) and (iv) are automatically satisfied once Filter (v) holds. In the exceptional case $V_2 / Q \equiv 0 \pmod{q}$, the two filters exclude at most $1 + e$ values of k among roughly Q admissible candidates. Therefore a random k yields a valid S_1 with probability at least $1 - (1/q + (1 + e)/Q)$. The values of $1/q + (1 + e)/Q$ for the three security levels are listed in Table 5. In our experiments, we typically succeed after at most two trials, i.e. $k = k_{\min} + 1$.

Table 5. Forgery results on KAZ-SIGN v1.6.4

Security Level	$1/q + (1 + e)/Q$	Time (s)	Success Rate
128-bit	$\approx 5.7 \times 10^{-14}$	0.001	100%
192-bit	$\approx 6.5 \times 10^{-30}$	0.001	100%
256-bit	$\approx 3.6 \times 10^{-47}$	0.001	100%

The range constraint $2^{L_{G_1 q Q \rho} - 1} \leq S_1 < G_1 q Q \rho$ induces a valid interval for k given by $k_{\min} \leq k \leq k_{\max}$, where $k_{\min} = \lceil (2^{L_{G_1 q Q \rho} - 1} - S_1^{(0)}) / M \rceil$ and $k_{\max} = \lfloor (G_1 q Q \rho - 1 - S_1^{(0)}) / M \rfloor$. Since $G_1 q Q \rho = M \cdot Q$ in the published parameters, we have $N_k = k_{\max} - k_{\min} + 1 \leq Q$, and N_k is on the order of Q . Thus the candidate set is enormous while only a constant number of k values are excluded.

We implement the forgery attack in SageMath and evaluate all three security levels. We average over 100 runs for each level. Table 5 reports the timing and success rates.

6 Conclusion

We demonstrate practical attacks against four cryptographic schemes submitted to the MySEAL 2.0 initiative: KAZ-KA v1.0, KAZ-KEM v1.0, KAZ-SIGN

v1.6.4, and KAZ-SIGN v2.0. We recover private keys for KAZ-KA, KAZ-KEM, and KAZ-SIGN v2.0, and we can perform universal signature forgeries for KAZ-SIGN v1.6.4 from the public key. Our attacks complete in under one second for all recommended security levels, namely 128, 192, and 256 bits.

The fundamental vulnerability of KAZ-KA v1.0, KAZ-KEM v1.0, and KAZ-SIGN v2.0 lies in the choice of the modulus N_j . By defining $N_j = \prod_{i=2}^{j+1} p_i$ as the product of consecutive small primes, the designers inadvertently make the group order $\varphi(N_j) = \prod_{i=2}^{j+1} (p_i - 1)$ extremely smooth. This smoothness has two critical consequences:

1. For KAZ-KA and KAZ-KEM, working modulo each small prime p_i reduces the DLP to groups of order at most $p_i - 1$. The two public key equations provide sufficient constraints to recover the two private key components via enumeration and table lookup.
2. For KAZ-SIGN v2.0, while the single public key equation is insufficient for direct key recovery, the signature scheme leaks linear relations involving the signing key and small ephemeral values. The size disparity between the ephemeral values ($\approx k$ bits) and $\varphi(N_j)$ ($\approx 3k$ bits) creates an HNP instance solvable via lattice reduction.
3. For KAZ-SIGN v1.6.4, the verification structure enables public-key signature forgery without recovering the private key.

We conclude that KAZ-KA v1.0, KAZ-KEM v1.0, KAZ-SIGN v1.6.4, and KAZ-SIGN v2.0 are insecure and should not be used. Any future designs based on similar principles must avoid smooth-order groups by using moduli with large prime factors. For KAZ-SIGN v2.0, the signature equations expose an HNP instance because the nonce values are much smaller than $\varphi(N_j)$, enabling lattice-based key recovery. For KAZ-SIGN v1.6.4, the verification structure leaves a large solution space that enables public-key signature forgery without recovering the private key.

References

1. Adrian, D., Bhargavan, K., Durumeric, Z., Gaudry, P., Green, M., Halderman, J.A., Heninger, N., Springall, D., Thomé, E., Valenta, L., VanderSloot, B., Wustrow, E., Zanella-Béguelin, S., Zimmermann, P.: Imperfect forward secrecy: How diffie-hellman fails in practice. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 5–17. ACM (2015). <https://doi.org/10.1145/2810103.2813707>
2. Ajtai, M.: Generating hard instances of lattice problems. In: Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. pp. 99–108 (1996)
3. Ariffin, M.R.K., Abu, N.A., Chang, K.C., Yuan, H.P., Ghafar, A.H.A., Mahad, Z., Cheon, L.M.: KAZ digital signature v1.6.4 (KAZ-SIGN v1.6.4). <https://mykripto.cybersecurity.my/index.php/files/101/AKBA-MySEAL/27/KAZ-SIGN-Updated.zip>, category: Digital Signature Scheme - Post-Quantum Digital Signature Scheme, updated version on September 2, 2025

4. Ariffin, M.R.K., Asbullah, M.A., Mahad, Z., Yusof, S.N., Ghafar, A.H.A.: KAZ-SIGN: Submission to NIST post-quantum cryptography standardization round 1. <https://csrc.nist.gov/Projects/pqc-dig-sig> (2023)
5. Ariffin, M.R.K., Nitaj, A., Abu, N.A., Mahad, Z., Asbullah, M.A., Ghafar, A.H.A.: KAZ digital signature v2.0 (KAZ-SIGN v2.0). <https://mykripto.cybersecurity.my/index.php/files/101/AKBA-MySEAL/26/KAZ-SIGN-20.zip>, category: Digital Signature Scheme - Post-Quantum Digital Signature Scheme, September 2, 2025
6. Ariffin, M.R.K., Nitaj, A., Abu, N.A., Mahad, Z., Asbullah, M.A., Ghafar, A.H.A.: KAZ key agreement v1.0 (KAZ-KA v1.0). <https://mykripto.cybersecurity.my/index.php/files/101/AKBA-MySEAL/28/KAZ-KA-Updated.zip>, category: Asymmetric Encryption Primitive – Key Agreement Scheme; updated version on September 2, 2025
7. Ariffin, M.R.K., Nitaj, A., Abu, N.A., Mahad, Z., Asbullah, M.A., Ghafar, A.H.A.: KAZ key encapsulation mechanism v1.0 (KAZ-KEM v1.0). <https://mykripto.cybersecurity.my/index.php/files/101/AKBA-MySEAL/25/KAZ-KEM-Updated.zip>, category: Asymmetric Encryption Primitive - Post-Quantum Key Encapsulation Mechanisms, updated version on September 2, 2025
8. Bernstein, D.J.: Official comment on KAZ-SIGN (2023), nIST PQC Digital Signature Schemes – Round 1 Additional Signatures, official comments thread
9. Boneh, D., Venkatesan, R.: Hardness of computing the most significant bits of secret keys in Diffie-Hellman and related schemes. In: Koblitz, N. (ed.) *Advances in Cryptology — CRYPTO '96*. pp. 129–142. Springer Berlin Heidelberg (1996)
10. Breitner, J., Heninger, N.: Biased nonce sense: Lattice attacks against weak ECDSA signatures in cryptocurrencies. In: Goldberg, I., Moore, T. (eds.) *Financial Cryptography and Data Security - 23rd International Conference, FC 2019, Frigate Bay, St. Kitts and Nevis, February 18-22, 2019, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 11598, pp. 3–20. Springer (2019). https://doi.org/10.1007/978-3-030-32101-7_1, https://doi.org/10.1007/978-3-030-32101-7_1
11. CyberSecurity-Malaysia: AKBA MySEAL 2.1. <https://mykripto.cybersecurity.my/index.php/services/myseal/myseal-category/akba-myseal/akba-myseal-2-1>, retrieved on January 14, 2026
12. CyberSecurity-Malaysia: Introduction to MySEAL. <https://mykripto.cybersecurity.my/index.php/services/myseal/introduction>, retrieved on January 14, 2026
13. CyberSecurity-Malaysia: National trusted cryptographic algorithm list. <https://www.cybersecurity.my/portal-main/services/myseal-overview>, retrieved on January 14, 2026
14. Howgrave-Graham, N., Smart, N.P.: Lattice attacks on digital signature schemes. *Des. Codes Cryptogr.* **23**(3), 283–290 (2001)
15. Jancar, J., Sedlacek, V., Svenda, P., Sys, M.: Minerva: The curse of ECDSA nonces (systematic analysis of lattice attacks on noisy leakage of bit-length of ECDSA nonces). *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2020**(4), 281–308 (2020). <https://doi.org/10.13154/tches.v2020.i4.281-308>
16. Kannan, R.: Minkowski’s convex body theorem and integer programming. *Mathematics of Operations Research* **12**(3), 415–440 (1987)
17. Kleinjung, T., Diem, C., Lenstra, A.K., Priplata, C., Stahlke, C.: Computation of a 768-bit prime field discrete logarithm. In: *Advances in Cryptology – EUROCRYPT 2017, Part I. Lecture Notes in Computer Science*, vol. 10210, pp. 185–201. Springer (2017). https://doi.org/10.1007/978-3-319-56620-7_7

18. Lenstra, A.K., Lenstra, H.W., Lovász, L.: Factoring polynomials with rational coefficients. *Mathematische Annalen* **261**(4), 515–534 (1982). <https://doi.org/10.1007/BF01457454>
19. MalaysiaGazette: Upm bangunkan teknologi kaz-sign. <https://malaysiagazette.com/2025/11/13/upm-bangunkan-teknologi-kaz-sign/> (Nov 2025), retrieved on January 14, 2026
20. Menezes, A., Okamoto, T., Vanstone, S.A.: Reducing elliptic curve logarithms to logarithms in a finite field. *IEEE Trans. Inf. Theory* **39**(5), 1639–1646 (1993). <https://doi.org/10.1109/18.259647>, <https://doi.org/10.1109/18.259647>
21. NEXCORE Tech: The kick-off poc initiative sandbox for development pqc migration. http://www.nexctech.com/news_detail_1/3.html (May 2025), retrieved on January 14, 2026
22. Nguyen, P.Q., Shparlinski, I.E.: The insecurity of the elliptic curve digital signature algorithm with partially known nonces. *Des. Codes Cryptogr.* **30**(2), 201–217 (2003). <https://doi.org/10.1023/A:1025436905711>, <https://doi.org/10.1023/A:1025436905711>
23. Pohlig, S., Hellman, M.: An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance. *IEEE Transactions on Information Theory* **24**(1), 106–110 (1978)
24. Pornin, T.: Deterministic usage of the digital signature algorithm (dsa) and elliptic curve digital signature algorithm (ecdsa). Request for Comments 6979, RFC Editor (Aug 2013), <https://www.rfc-editor.org/rfc/rfc6979>
25. Shanks, D.: Class number, a theory of factorization, and genera. In: *Proc. Symp. Math. Soc.*, 1971. vol. 20, pp. 415–440 (1971)
26. team, T.K.: KAZ PQC demo. <http://110.4.41.112:8118/>, retrieved on January 14, 2026
27. WeakDH Team: The logjam attack. <https://weakdh.org/logjam.html> (2015), accessed January 2026